

키팩트 F1 병목 해소를 위한 결정론 계층 설계

약관 4 종 QA 파이프라인 개선 보고 · Qwen2.5-3B / Colab T4

2026

총점 추이

80.76 → 90.52

1 차 완성 대비 +9.76 점 (예상)

MRR

1.0000

20.0 / 20

키팩트 F1

0.6741

20.2 / 30

LLM 판정

97.00

48.5 / 50

손실의 72% 가 키팡트 F1 에서 발생한다

1 차 완성 (80.764) 기준 배점별 손실 분해 . 검색은 이미 포화 상태이므로 개선 여지가 없다 .

MRR

0.00

점 손실

20.000 / 20

키팡트 F1

13.94

점 손실

16.063 / 30

LLM 판정

5.30

점 손실

44.700 / 50

진단

MRR 은 공개 10 문항 전부 rank 1 로 이미 상한이다 .

3B 모델의 자유생성만으로는 원문 어휘 재현 (precision) 과 근거 누락 방지 (recall) 를 동시에 보장할 수 없다 .

따라서 개선 대상은 검색이 아니라 **생성 앞뒤의 결정론 계층**이다 .

채점식 역설계가 모든 판단의 전제다

```
# 공개 10 문항 채점값 재현 실험
keyfact_f1 == multiset_token_f1(
    answer.split(), gold_keyfact.split()
)
# 재현 오차 MAE 0.026
```

확정된 사실

- 키팍트 F1 은 어절 단위 멀티셋 토큰 F1 이다 .
- 골드 키팍트는 약관 원문 문장을 거의 그대로 옮긴 것이다 .
- 의역은 의미가 맞아도 토큰이 갈려 손해다 . 원문 인용이 F1 상한에 가장 가깝다 .

여기서 도출된 답변 요건 3 가지

- a** 원문 어휘를 그대로 옮긴다
F1 precision + LLM 근거성 25%
- b** 필요한 근거를 빠짐없이 담는다
F1 recall + LLM 완결성 20%
- c** 그 외 토큰을 최소화한다
F1 precision + LLM 명료성 15%

접근 방향 프롬프트로 유도하되 , 유도가 실패해도 코드가 교정한다 . 생성 품질에 요건 3 가지를 맡기지 않는다 .

원본에 있는 구조를 버린 것이 결함의 단일 원인이다

이전 방식

BeautifulSoup 으로 DOM 구성 후 `get_text()` 로 즉시 평문화 . 항 · 호 경계를 정규식으로 재구성 .

실제 HTML

`tit_subject( 조 ) · list_1depth( 항 ) · list_2depth( 호 )` 클래스로 구조를 이미 명시하고 있었다 .

구조 폐기에서 파생된 결함 5 건

문항	관측된 결함	원인
P03	호 5 번 유실 + 광고 문장 혼입	항에 종속된 호 목록이 평문화 후 다음 항과 뒤섞임
P04	무관 호 목록 유입	② 항과 ③ 항의 경계 소실
P02	엉뚱한 항 선택	항 단위 후보가 부정확해 정답 항을 특정하지 못함
P10	주체 왜곡	생성 모델이 원문을 풀어 쓰며 왜곡
구조	조 / 장 혼동 위험	정규식이 장 제목까지 조로 오인할 여지

표 1. 관측된 결함과 원인 . 5 건 전부 단일 지점에서 파생됐다 .

결론 임계값 조정으로는 풀리지 않는다 . 개선 지점은 선별기가 아니라 취득 단계의 정보 손실이다 .

클래스 기반 DOM 파싱으로 전환한다

```
class Provision(BaseModel):
    label: str = ""; text: str = ""
    items: List[str] = Field(default_factory=list)

def extract_structured_articles(container):
    articles, current, consumed = [], None, set()
    for node in container.find_all(True):
        if id(node) in consumed:      # 중복 부착 방지
            continue
        if _has_class(node, ARTICLE_HEADING_CLASS):
            heading = _parse_article_heading(node)
            current = StructuredArticle(**heading)
        elif _has_class(node, PARAGRAPH_LIST_CLASS):
            current.provisions.append(      # 항
                _build_provision(node, consumed))
        elif _has_class(node, ITEM_LIST_CLASSES):
            current.provisions[-1].items.extend( # 호
                _read_items(node, consumed))
```

1 조 경계 판정

tit_subject 클래스와 "제 N 조" 패턴을 동시에 만족할 때만 조로 인정한다. 시행일자 같은 비조 요소를 배제한다.

2 호 귀속 처리

호가 항 li 안에 중첩된 경우 (카카오계정 약관) 와 형제로 배치된 경우 (위치정보 약관) 를 모두 직전 항에 귀속시킨다.

3 중복 부착 방지

consumed 집합으로 처리한 노드를 추적한다. find_all() 순회 특성상 같은 호가 다음 항에 다시 붙는 것을 막는다.

하위 호환 기존 스키마는 필드명·타입을 바꾸지 않고 structured, provisions 필드만 추가했다. parse_articles 는 구조가 있으면 그대로 쓰고, 없을 때만 기존 정규식 경로로 폴백한다.

규칙과 LLM 의 분담 기준을 정한다

위양성 위험이 0에 가까우면 규칙, 의미 판단이 필요하다면 LLM. 이 분담이 임계값 튜닝 부담을 줄인다.



규칙 담당 : 통계적 근거가 확정된 것만

```
def classify_provision(text: str) -> str:
    if _match(text, PREAMBLE_PATTERNS):
        return "안내문" # 골드 26 건 중 0 건
    if _match(text, DEFINITION_PATTERNS):
        return "정의문" # 정의 질문일 때만 유지
    return "규범문"
```

골드 키팩트 26 건 중 안내문 마커가 붙은 것은 0 건, 과다포함 7 건 중 3 건은 안내문이었다. 규범적 효력이 없으므로 무조건 제외해도 안전하다.

LLM 담당 : 표층 신호로 풀리지 않는 것

```
UNIT_SELECTION_PROMPT = (
    " 질문에 답하려면 반드시 필요한 문장의 "
    " 번호만 심표로 구분해 출력하십시오 .\n"
    "- 같은 조라도 무관하면 고르지 않습니다 \n"
    "- 번호 외에는 출력하지 않습니다 . 예 : 2,5"
) # max_new_tokens=24, 실패 시 None 반환
```

P01에서 유사 문장 3개가 바이그램 0.28~0.75로 뭉쳐 표층 어휘로는 구분이 원리적으로 불가능했다. 판단 주체 자체를 바꿨다.

선별 실패는 답변 실패가 아니다 모델 없음·파싱 불가·범위 초과 시 None을 반환해 호출부가 표층 점수로 풀백한다.

생성 이후 정련 5 단계로 환각과 누락을 교정한다



필터(3) 를 수리(4) 보다 먼저 돌리는 이유: 보충한 원문 문장이 필터에 잘려나가지 않게 하기 위해서다.

```
def groundedness_filter(sents, context, evidence, cfg):
    for core in sents:
        # 기준 1: 조문에 실재하는가 ( 환각 차단 )
        if bigram_overlap(core, context) < cfg.grounded_threshold:
            continue # 0.42
        # 기준 2: 선별 근거 안에 있는가 ( 주제 이탈 차단 )
        if evidence and bigram_overlap(core, evidence) \
            < cfg.evidence_threshold: # 0.30
            continue
        yield core
```

환각 차단

조문 전체와 대조한다 . P09 의 " 회사가 모든 권한을 가진다 " 창작 (accuracy 3/10) 을 잡는다 .

주제 이탈 차단

LLM 이 의미로 고른 근거 단위와 대조한다 . 조문에는 있으나 이 질문엔 불필요한 문장 (P01·P05) 을 잡는다 .

커버리지 수리는 반대로 문턱을 없앴다 . key_units 는 이미 선별 단계가 걸러낸 목록이므로 같은 약한 신호로 재검증하면 정답 문장을 지운다 . P02 의 " 즉시 복구 노력 " 은 질문과 겹침이 0.09 에 불과하지만 골드 키팩트에 필요했다 . 관련도 문턱은 선별 단계 하나에서만 적용한다 .

모든 변경이 공개 10 문항의 감점 사례에서 역산됐다

문항	관찰된 실패	대응 장치	도입
P01	유사 문장 3 개가 바이그램 0.28~0.75 로 뭉쳐 구분 불가	LLM 의미 선별	U-3
P02	정답 조 내 문장 누락 , 질문과 겹침 0.09	커버리지 수리 + 재검증 제거	U-2/3
P03	5 개 항목 중 2 개만 출력 , 이후 정의문 중복	열거형 추출식 + enumeration_satisfied	U-2/3
P05	조문에는 있으나 무관한 문장 포함	evidence_threshold 필터	U-3
P08	" 제 N 조에 따르면 ," 도입부 재진술	META_PREFIX_PATTERNS	U-2
P09	원문과 정반대 문장 창작 , accuracy 3/10	groundedness_filter	U-2
P10	앞부분 어휘가 겹쳐 누락 미검출	꼬리 겹침 min() 동시 요구	U-2

표 2. 문항별 실패와 대응 장치의 1:1 매핑.

채점기 재현이 선행됐다 MAE 0.026 으로 채점값을 재현한 것이 모든 판단의 전제다 . " 키팍트 F1 은 어절 멀티셋 F1" 이라는 사실이 확정되지 않았다면 원문을 그대로 옮긴다는 방향 자체가 성립하지 않는다 .

임계값 조정

필드	U-2	U-3
key_unit_top_n	4	3
key_unit_min_score	0.15	0.20
evidence_threshold	-	0.30

표 3. LLM 선별이 정밀해져 후보 수를 줄여도 recall 이 유지된다 . 표층 점수는 폴백 전용이 되어 보수적으로 상향했다 .

근거 슬롯 다양화

MRR 은 정답이 처음 등장하는 순위만 보고 오답 슬롯에 페널티가 없다 . 검증된 상위 2 개는 보존하고 , 이하 슬롯은 미등장 문서를 우선 채운다 . 정답이 상위에 있으면 동일하게 1.0, 없으면 0 점 대신 0.25 를 확보하는 손실 없는 보험이다 .

생성 호출 2 배를 감수한 근거는 세 가지다

질문 1 건당 생성 호출

1 → 2 회

1 추가 모델 적재 없음

서빙 모델을 선별기로 검용하므로 T4 VRAM 부담이 0 이다. 별도 분류 모델을 올렸다면 bge-m3 fp16 임베딩 + 리랭커와 합쳐 VRAM 이 빠듯해진다.

2 출력이 짧음

선별 출력은 번호 몇 개뿐이라 max_new_tokens=24 로 끝난다. 답변 생성 448 토큰에 비하면 부수적이다.

3 채점 배분

지연시간은 프로토콜 (requests_per_run=12, concurrency=2) 통과 여부의 문제이고, 점수는 MRR 20 + F1 30 + LLM 50 에서 나온다.

모든 신규 경로에 폴백을 둔다

```
try:    select_units_by_llm(...)        # 의미 선별
except: question_relevance_score(...)  # 표층 점수
try:    refine_answer(...)              # 정련
except: generated_answer                # 생성 그대로
if _RERANKER is None: bigram_overlap(...) # 재순위 없이
if not has_rank_bm25: dense_search(...)  # BM25 없이
```

품질 장치를 늘리면 실패 지점도 늘어난다. 어느 하나가 죽어도 답변이 나오는 구조를 유지했다.

유지한 속도 최적화

3B fp16 채택 T4(sm75) 에서 bnb 4bit 역양자화 오버헤드가 커 7B-4bit 보다 지연시간에서 유리하다.

사전 양자화 체크포인트 로드 타임 양자화 (7B 15GB) 를 4bit 저장 가중치 (5GB) 다운로드로 대체해 부팅을 단축했다.

미세조정 → 인덱싱 순서 학습 시점에 임베딩 · 재순위 모델이 아직 안 올라가 있어 VRAM 최고점이 낮다.

구조 파싱 전환이 단일 최대 개선폭을 만들었다

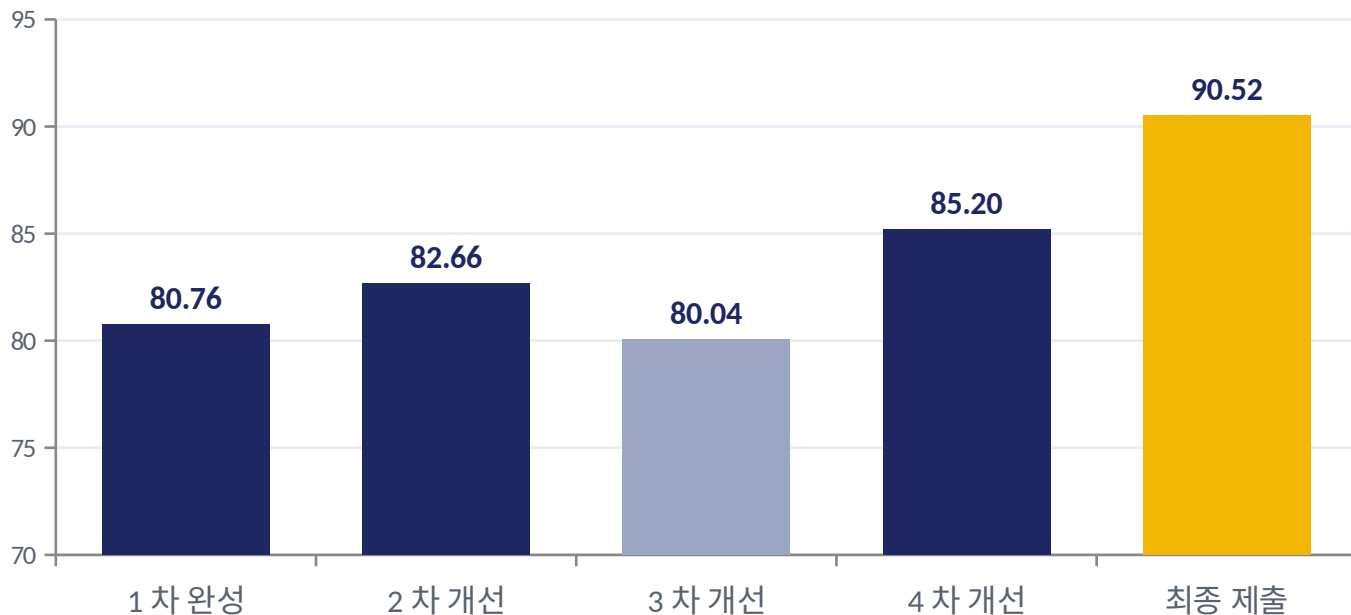


그림 1. 버전별 총점(100 점 만점). 3 차는 임계값 튜닝이 다른 문항을 퇴보시킨 사례다.

튜닝 대 구조 임계치 튜닝은 관측된 실패 패턴에 값을 맞추는 방식이라 한 차례는 다른 문항을 퇴보시켰다 (P04 -1.29 점). 구조 파싱은 4 종 약관 HTML 마크업이라는 고정된 사실에 근거하므로 비공개 30 문항에도 문항 수와 무관하게 그대로 적용된다.

구조 파싱 적용 전후

키팍트 F1 (평균)

0.5568 → **0.6741** +21.1%

키팍트 점수 (30 점)

16.70 → **20.22** +21.1%

총점

85.20 → **88.72** +4.1%

문항별 개선폭

P08 0.316 → 0.806 +155%

P03 0.333 → 0.764 +129%

P01 0.561 → 0.865 +54%

P07 0.402 → 0.370 -8%

병목이 취득·파싱에서 선별 단계로 이동했다

구조 파싱은 "어떤 항이 존재하는가"를 정확히 만드는 작업이지 "그중 무엇을 답으로 고를 것인가"를 정하지 않는다.

구조적 한계

P07

골드 사실과 무관한 내용이 원문 한 문장 안에 공존한다. 문장 단위 분해로는 해소 불가능하다.

선별기 오선택

P02 · P09

정답 문장이 후보에는 있었으나 선별기가 다른 문장을 골랐다. 후보 정확도가 아닌 선별 정확도의 문제다.

후처리 미검출

P10

생성 모델이 만든 비문이 정상 종결되어 drop_dangling_tail 이 발동하지 않는다.

디코딩 부작용

P03

prompt_lookup_num_tokens=10 이 번호 목록의 반복 n-gram 에서 내용을 건너뛴다.

추가 개선 방향

01

cross-encoder 재순위를 선별기로 승격

MRR 1.0 을 이미 달성한 모델이므로 LLM 선별기보다 신뢰도가 높다.

02

추출 우선 생성 경로

선별 단위가 필요한 사실을 전부 덮는 문항은 생성 없이 원문을 그대로 낸다.

03

단일 변수 격리 재실험

prompt_lookup_num_tokens=0 으로 두고 P03 붕괴 원인을 단독 검증한다.